Oliver Krauss

Modern Code - 2D Arrays & Grid Structures

ModernCode | MTD.ba VZ SS25

Hagenberg 13.01.2026

HAGENBERG | LINZ | STEYR | WELS

FH OBERÖSTERREICH

UNIVERSITY OF APPLIED SCIENCES UPPER AUSTRIA

2D Arrays & Grid Structures

- **The World is not just a line**: Many data structures are 2-dimensional
- **Grids are everywhere**: Games, Images, Spreadsheets, Maps
- **Connection to what you know**:
  - 1D Arrays: A list of items
  - Nested Loops: A loop inside a loop
- **Today's Goal**: Learn to represent and manipulate grid-based data

Before We Begin: Think About This

How would you represent a Chess board or a Tic-Tac-Toe grid in code?

- `int cell1, cell2, ... cell64;`? (Too many variables!)
- `int[] board = new int[64];`? (Math gets complicated: index = row * 8 + col)
- **Is there a better way?**

Bridge: From 1D to 2D

1D Array (The Line)

- One index: `arr[i]`
- Good for lists, sequences.

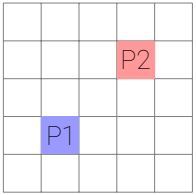
2D Array (The Grid)

- Two indices: `arr[row][col]`
- Good for tables, boards.

The World is 2D (and 3D)

Examples of Grid Data:

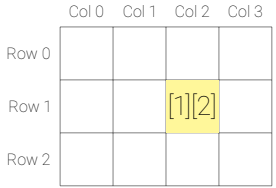
- **Spreadsheets**: Rows and Columns
- **Pixel Art**: Grid of colors
- **Cinema Seating**: Row A, Seat 5
- **Maps**: Latitude and Longitude
- **Game Boards**: Chess, Checkers, Minesweeper



A simple grid world

Mental Model: The Row-Column Grid

- **Concept**: Think of a 2D array as a table with Rows and Columns
- **Indices**: We need **two** numbers to find an item: `[row][col]`
- **Analogy**: A Hotel
  - First number: Floor (Row)
  - Second number: Room Number (Column)



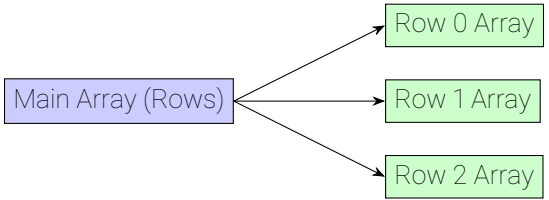
Accessing Data:

`matrix[1][2]`

Row 1, Column 2

Under the Hood: Array of Arrays

- Java doesn't strictly have "2D Arrays"
- **Reality**: Java has "Arrays of Arrays"
- `int[][] matrix` is an array where each element is a reference to another `int[]` array

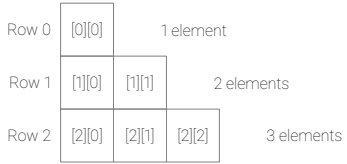


Note: Since each row is a separate array, rows can have different lengths!

Jagged Arrays: Rows of Different Lengths

- Since a 2D array is an "Array of Arrays", each row can have a different length
- **Use Case**: Storing data that isn't a perfect grid

Jagged Array (Pascal's Triangle style)




```
int[][] triangle = new int[3][];
triangle[0] = new int[1]; // Row 0: 1 column
triangle[1] = new int[2]; // Row 1: 2 columns
triangle[2] = new int[3]; // Row 2: 3 columns
```

Memory Model: 2D Arrays on Stack and Heap

```
1   static void example() {
2       int[][] matrix = new int
        [3][4];
3       matrix[1][2] = 42;
4   }
```

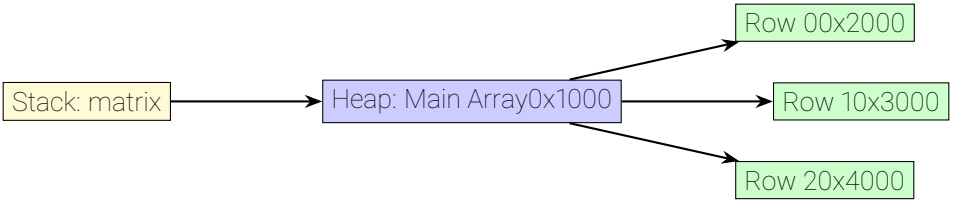
Stack Variables

Variable	Value
matrix	0x1000

Heap Objects

Address	Object
0x1000	Main Array: [ref0, ref1, ref2]
0x2000	Row 0: [0, 0, 0, 0]
0x3000	Row 1: [0, 0, 42, 0]
0x4000	Row 2: [0, 0, 0, 0]

Memory Model: Visual Representation



- **Stack**: Variable `matrix` holds a reference (address)
- **Heap**: Main array object contains references to row arrays
- **Heap**: Each row array contains the actual integer values
- **Note**: All array objects live on the heap, just like 1D arrays!

Step-by-Step: How `matrix[1][2]` is Resolved

```
1   int[][] matrix = new int[3][4];
2   int value = matrix[1][2];  // How does this work?
```

Step 1: `matrix[1]`

- Accesses the main array at index 1
- Returns a reference to Row 1 array
- Result: Reference to **`int[]`** array at address 0x3000

Step 2: `[2]` on the result

- Takes the Row 1 array reference from Step 1
- Accesses that array at index 2
- Returns the integer value stored at that position
- Result: The value 42 (or 0 if not yet set)

Think of it as: `(matrix[1])[2]` - first get the row, then get the column within that row.

Declaring and Creating

Syntax

```
Type[][] name = new Type[rows][cols];
```

```
1   // Create a 3x4 grid (3 rows, 4 columns)
2   int[][] matrix = new int[3][4];
```

Accessing Elements

```
1   // Accessing elements
2   matrix[0][0] = 5;       // Top-left
3   matrix[2][3] = 10;      // Bottom-right (Row 2, Col 3)
4   int x = matrix[1][2];  // Read from Row 1, Col 2
```

Common Pitfall: Row vs Column

It is always `[ROW][COLUMN]`.

Think **R.C.** (Remote Control, or Race Car) to remember the order.

Initialization

Direct Initialization: You can initialize a 2D array with values directly using nested curly braces.

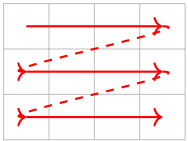
```
int[][] grid = {
    {1, 2, 3},    // Row 0
    {4, 5, 6},    // Row 1
    {7, 8, 9}     // Row 2
};

// grid[1][2] is 6
// grid.length is 3 (number of rows)
// grid[0].length is 3 (number of columns in row 0)
```

Traversing: Row-Major (Standard)

Concept: Row by Row.

- Outer Loop: `row`
- Inner Loop: `col`
- **Cache Friendly** (Contiguous memory)

```
1    for (int r = 0; r < rows; r++) {
2        for (int c = 0; c < cols; c++) {
3            print(matrix[r][c]);
4        }
5        println();
6    }
```



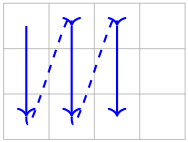
Row-Major

Traversing: Column-Major

Concept: Column by Column.

- Outer Loop: `col`
- Inner Loop: `row`
- **Use Case**: Vertical checks (Connect 4)

```
1    for (int c = 0; c < cols; c++) {
2        for (int r = 0; r < rows; r++) {
3            print(matrix[r][c]);
4        }
5    }
```



Column-Major

Pattern: Sum of All Elements

```
1   int totalSum = 0;
2
3   for (int r = 0; r < matrix.length; r++) {
4       for (int c = 0; c < matrix[r].length; c++) {
5           totalSum += matrix[r][c];
6       }
7   }
```

Variations:

- Find Max/Min value
- Count occurrences of a specific value
- Check if grid contains a value (Search)

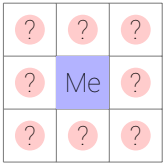
Pattern: Neighbor Checking (The Minesweeper Problem)

Problem: For a cell at `[r][c]`, check its 8 neighbors.

- Top-Left: `[r-1][c-1]`
- Top: `[r-1][c]`
- ... and so on.

Challenge: Boundary Checks!

- If `r=0`, we can't check `r-1` (IndexOutOfBounds)
- Must check `isValid(r, c)` before accessing.

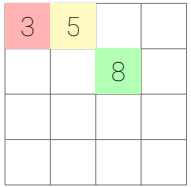


Boundary Cases: Corners, Edges, and Interior

Three Types of Cells:

- **Corner**: 3 neighbors (e.g., top-left)
- **Edge**: 5 neighbors (e.g., top row, not corner)
- **Interior**: 8 neighbors (middle of grid)

Solution: Always validate before accessing!



Neighbor counts

Helper Method: `isValid()`

The Standard Pattern: Create a helper method to check boundaries.

```
1  // Check if (r, c) is a valid position in the grid
2  static boolean isValid(int[][] grid, int r, int c) {
3      // Check row bounds
4      if (r < 0 || r >= grid.length) return false;
5      // Check column bounds (use first row's length)
6      if (c < 0 || c >= grid[0].length) return false;
7      return true;
8  }
```

Usage: Always call `isValid()` before accessing neighbors!

Complete Neighbor Checking Example

Count neighbors in a Minesweeper-style grid:

```
1     int countNeighbors(int[][] grid, int r, int c) {
2         int count = 0;
3         // Check all 8 directions
4         int[] dr = {-1, -1, -1,  0, 0,  1, 1, 1}; // row deltas
5         int[] dc = {-1,  0,  1, -1, 1, -1, 0, 1}; // col deltas
6
7         for (int i = 0; i < 8; i++) {
8             int newR = r + dr[i];
9             int newC = c + dc[i];
10            // Only check if valid position
11            if (isValid(grid, newR, newC) && grid[newR][newC] == 1) {
12                count++;
13            }
14        }
15        return count;
16    }
```

Key: The `isValid()` check prevents `ArrayIndexOutOfBoundsException`!

Common Operations: Transpose

Transpose: Swap Rows and Columns.

```
int[][] transpose(int[][] matrix) {
    int h = matrix.length; int w = matrix[0].length;
    int[][] result = new int[w][h]; // Dimensions swapped!

    for (int r = 0; r < h; r++) {
        for (int c = 0; c < w; c++) {
            result[c][r] = matrix[r][c]; // Swap indices
        }
    }
    return result;
}
```

Common Operations: Flood Fill

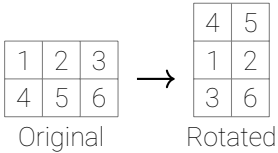
Concept: Filling a region (like Paint Bucket tool).

- Start at a pixel.
- Spread to neighbors if they are the same color.
- **Preview**: This uses **Recursion**!

Common Operations: Rotate Grid 90 Degrees

Concept: Rotate a grid clockwise by 90 degrees.

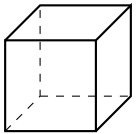
```
1   int[][] rotate90(int[][] grid) {
2       int rows = grid.length;
3       int cols = grid[0].length;
4       int[][] rotated = new int[cols][rows];
5
6       for (int r = 0; r < rows; r++) {
7           for (int c = 0; c < cols; c++) {
8               // Rotate: (r,c) -> (c, rows-1-r)
9               rotated[c][rows - 1 - r] = grid[r][c];
10          }
11      }
12      return rotated;
13  }
```



Key Insight: Dimensions swap! A 3x2 grid becomes 2x3 after rotation.

Going Deeper: 3D Arrays

- **Concept**: An array of 2D arrays.
- **Visual**: A Cube, or a Stack of Pages (Book).
- **Syntax**: `int[][][] cube = new int[depth][rows][cols];`
- **Access**: `cube[z][y][x]` (or `[layer][row][col]`)



Applications of 3D Arrays

- **3D Space**: Voxel engines (like Minecraft chunks)
- **Time**: A sequence of 2D board states (History of a chess game)
- **Color Images**:
  - 2D Grid of Pixels
  - Each Pixel has 3 values: Red, Green, Blue
  - Structure: `[height][width][3]`

Matrices & Vectors: The Math Connection

Computer Science $\leftrightarrow$ Mathematics

- **1D Array $\approx$ Vector** $(v \in \mathbb{R}^n)$
- **2D Array $\approx$ Matrix** $(M \in \mathbb{R}^{m \times n})$

Why it matters?

- **Graphics**: Rotating 3D objects uses Matrix Multiplication
- **AI/ML**: Neural Networks are giant Matrix operations
- **Physics**: Velocity and Position are Vectors

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}$$

A 2×3 Matrix

$$\vec{v} = \begin{pmatrix} x \\ y \\ z \end{pmatrix}$$

A Vector

Matrix Operations: Vector Addition

Vector Addition (Element-wise):

- Add corresponding elements from two vectors
- Result is a new vector of the same size

```
// Vector Addition: C = A + B
for (int i = 0; i < n; i++) {
    result[i] = vectorA[i] + vectorB[i];
}
```

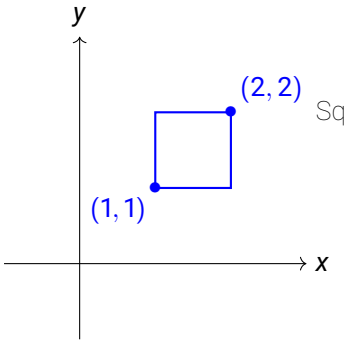
Matrix Operations: Multiplication

Matrix Multiplication (Conceptual): Row of A · Column of B (Dot Product). Requires **Triple Nested Loop**!

```
// Matrix Multiplication: C = A * B
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        for (int k = 0; k < common; k++) {
            result[i][j] += A[i][k] * B[k][j];
        }
    }
}
```

The "Object" in 2D Space

- **What is an Object?** A shape (Square, Triangle, Character) is just a list of **Points** (Vectors).
- **To Move/Transform the Object**: We apply the same math operation to **every point**.

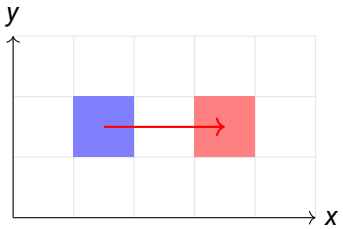


Transformation 1: Translation (Moving)

- **Goal**: Move the object by (d_x, d_y).
- **Math**: Vector Addition.
- **Formula**: $v_{new} = v_{old} + \text{translation}$

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} x \\ y \end{pmatrix} + \begin{pmatrix} 2 \\ 0 \end{pmatrix}$$

Move Right by 2

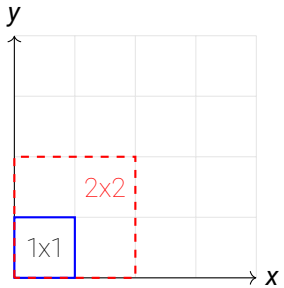


Transformation 2: Scaling (Resizing)

- **Goal**: Make the object bigger or smaller.
- **Math**: Matrix Multiplication.
- **Matrix**: $\begin{pmatrix} s_x & 0 \\ 0 & s_y \end{pmatrix}$

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix}$$

Double the size $(2x, 2y)$

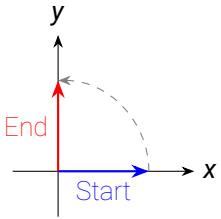


Transformation 3: Rotation (Spinning)

- **Goal**: Rotate around the origin $(0, 0)$.
- **Math**: Matrix Multiplication.
- **Matrix**: $\begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$

$$\begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

Rotate **90°** Counter-Clockwise



Combining Transformations

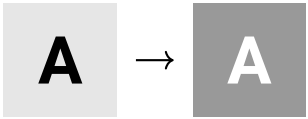
- **Power of Matrices**: We can combine operations!
- To Scale AND Rotate: Multiply the matrices.
- $M_{combined} = M_{rotate} \times M_{scale}$
- This is how game engines move thousands of objects efficiently.

Deep Dive: Image Processing

Concept: An image is just a grid of pixels.

- Grayscale: 0 (Black) to 255 (White)
- **Task**: Invert the image (Negative)

```
// Invert Image
for (int r = 0; r < height; r++) {
    for (int c = 0; c < width; c++) {
        int original = image[r][c];
        image[r][c] = 255 - original;
    }
}
```

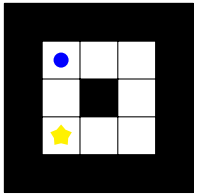


Deep Dive: Mazes & Game Logic

Grid Representation:

-- 0 = Path, 1 = Wall, 2 = Player, 9 = Goal

```
int[][] maze = {
    {1, 1, 1, 1, 1},
    {1, 2, 0, 0, 1},
    {1, 1, 1, 0, 1},
    {1, 9, 0, 0, 1},
    {1, 1, 1, 1, 1}
};

boolean isValidMove(int r, int c) {
    // Check boundaries
    if (r < 0 || r >= rows || c < 0 || c >= cols)
      return false;
    // Check wall
    if (maze[r][c] == 1) return false;

    return true;
}
```

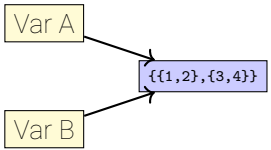


Pitfall: Shallow Copy vs Deep Copy

Warning

Arrays are **Objects**. Assigning one variable to another copies the **Reference**, not the Data!

```
1  int[][] A = {{1, 2}, {3, 4}};
2  int[][] B = A; // Shallow
       Copy!
3
4  B[0][0] = 99;
5  // A[0][0] is NOW 99!
6  // B points to SAME memory as
       A.
```



Methods & 2D Arrays

Modular Code: Pass arrays to methods to keep your code clean.

```
// Method accepts a 2D array as a parameter
public static void printGrid(int[][] data) {
    for (int r = 0; r < data.length; r++) {
        for (int c = 0; c < data[r].length; c++) {
            System.out.print(data[r][c] + " ");
        }
        System.out.println();
    }
}
```

Recursion & Grids: Maze Solving

Concept: Recursion is great for exploring grids (Pathfinding).

```
boolean solveMaze(int r, int c) {
    // 1. Base Case: Out of bounds or Wall
    if (!isValid(r, c) || isWall(r, c)) return false;
    // 2. Base Case: Reached Goal!
    if (isGoal(r, c)) return true;

    // 3. Recursive Step: Try all 4 directions
    markVisited(r, c);
    if (solveMaze(r+1, c)) return true; // Down
    if (solveMaze(r, c+1)) return true; // Right
    // ... Up and Left

    return false; // Backtrack
}
```

Common Mistakes: Overview

Four common pitfalls when working with 2D arrays:

- Row/Column confusion (Cartesian vs Array indexing)
- Index out of bounds errors
- Nested loop variable name conflicts
- Shallow copy vs deep copy confusion

Why this matters: These mistakes cause runtime crashes, incorrect results, and hard-to-debug issues.

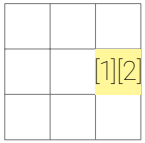
Common Mistake 1: Row/Column Confusion

The Problem

Thinking in Cartesian coordinates (x, y) instead of array indexing (row, col).

Wrong:

```
// Thinking: x=2, y=1
matrix[2][1] = 5;
// This is Row 2, Col 1!
```



Row 1, Col 2

Right:

```
// Think: Floor 1, Room 2
matrix[1][2] = 5;
// Row 1, Column 2
```

Mnemonic: **R.C.** = Remote Control or Race Car (Row comes first, then Column)

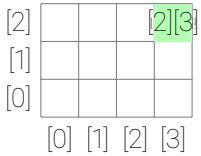
Common Mistake 2: Index Out of Bounds

The Problem

Using array dimensions instead of valid indices (0 to length-1).

Wrong:

```
1    int[][] matrix = new int[3][4];
2    // CRASH! Index 3 doesn't exist
3    matrix[3][4] = 5;
```



Valid indices: 0-2 rows, 0-3 cols

Right:

```
1    int[][] matrix = new int[3][4];
2    // Valid: indices 0 to 2 for rows
3    // Valid: indices 0 to 3 for cols
4    matrix[2][3] = 5; // Last element
```

Remember: For array of size `n`, valid indices are `0` to `n-1`

Common Mistake 3: Nested Loop Variable Names

The Problem

Using the same variable name in nested loops causes compilation errors or unexpected behavior.

Wrong:

```
    for (int i = 0; i < rows; i++) {
        for (int i = 0; i < cols; i++) {
            // ERROR: Variable 'i' already defined!
            matrix[i][i] = 0;
        }
    }
```

Right:

```
    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            matrix[r][c] = 0;
```

Best Practices:

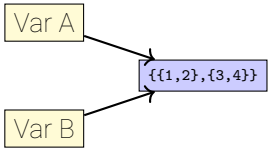
- Use `r, c` for row/column
- Or `i, j` if you prefer
- **Never** reuse the same variable name
- Descriptive names make code clearer

Common Mistake 4: Shallow Copy (Enhanced)

The Problem

Assigning one 2D array variable to another only copies the reference, not the data!

```
int[][] A = {{1, 2}, {3, 4}};
int[][] B = A; // Shallow Copy!

B[0][0] = 99;
// A[0][0] is NOW 99!
// Both point to SAME data
```



Both reference same object

Solution: Create a deep copy if you need independent arrays:

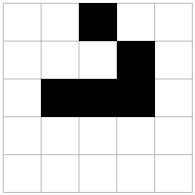
```
int[][] B = new int[A.length][A[0].length];
for (int r = 0; r < A.length; r++) {
    for (int c = 0; c < A[r].length; c++) {
        B[r][c] = A[r][c]; // Copy each value
    }
}
```

Challenge: Conway's Game of Life

The Rules of Life:

- Grid of cells: Alive (1) or Dead (0).
- **Underpopulation**: < 2 neighbors → Dies.
- **Overpopulation**: > 3 neighbors → Dies.
- **Survival**: 2 or 3 neighbors → Lives.
- **Reproduction**: Exactly 3 neighbors → Becomes Alive.

Why?: It's Turing Complete! You can build a computer inside it.



The "Glider"

Summary: 2D Arrays & Grid Structures

Core Concepts

- 2D Arrays = Arrays of Arrays
- Row-Column indexing: `[row][col]`
- Memory: Stack variables reference heap objects
- Rectangular vs Jagged arrays

Traversal Patterns

- Row-major (standard, cache-friendly)
- Column-major (vertical processing)
- Nested loops for systematic access

Common Operations

- Transpose, Rotate, Find Max/Min
- Count occurrences, Sum elements

Connections to Previous Sessions

- **Session 06**: 1D arrays → 2D arrays
- **Session 03**: Nested loops for traversal
- **Session 04**: Methods with 2D array parameters
- **Session 10**: Recursion for maze solving, flood fill
- **Session 07**: Grid-based search algorithms

Common Mistakes

- Row/Column confusion (use R.C. mnemonic)
- Index out of bounds (0 to length-1)
- Nested loop variable name conflicts
- Shallow copy vs deep copy

Key Takeaways

- 2D arrays model grid-based problems